University of Applied Sciences and Arts
Northwestern Switzerland
School of Computer Science
Institute for Mobile and Distributed Systems (IMVS)

Master of Science in Engineering (MSE)

Project P8

Enhancing Web Accessibility Standards

Implementation of User-Centric Customization Interfaces and CSS-based Contrast Optimization

---

Author: **Florian Schnidrig**

Advisor: Prof. Dierk König

Profile: Computer Science

Date: Summer 2026 (TBD)

Contents

3. Introduction

The previous project, *Accessibility on the Web – Where we Stand* [1], examined the fundamental principles of digital inclusion within modern web applications. It explored how semantic HTML serves as the backbone for accessibility, provided an overview of both common and specialized native elements, and clarified the roles of ARIA and WCAG. Furthermore, it offered foundational guidance on accessibility debugging, testing, and the practical use of screen readers.

Building upon that foundation, this project addresses essential aspects of accessibility that, while perhaps less technical in a traditional coding sense, are no less vital for an inclusive digital world. While our previous focus remained largely on the structural and technical "under the hood" mechanics, we have yet to explore the visual design landscape. These design choices are far from purely aesthetic; they play a decisive role in determining whether a web application is a welcoming space or a digital dead end for many users.

4. Theory

4.1. Testing Accessibility

4.2. CSS Media Queries

CSS Media Queries allow developer to apply different styles based on characteristics of a device or environment displaying a web page. This is often used to create a responsive web page that uses different stylings, based on screen width.

The structure of such a media query looks like the following:

```
@media [not] media-type and (media-feature: value) and (media-feature:
value) {
  /* CSS rules to apply */
}
```

For web development the `screen` media-type is mainly of interest, but it is also possible to set it to `print` for targeting printers. The default value is `all`, representing both independent options and is suitable for all devices.

Media-feature is where it gets interesting as there are currently over 40 options available to listen and react to. For example, both max-width and max-height, as well as their opposites min-width and min-height, are widely used to create responsive web sites with different appearances optimized for different devices. Some of these features consider aspects of accessibility and user preferences which make them essential to create an accessible web page considering individual needs of different users, but they are not as widely used yet as they should. The following media-feature values should be considered when building web applications that should include everyone. [2]

4.2.1. Reduced Motion

The `prefers-reduced-motion` media-feature indicated if a user has enabled a setting on their device to minimize the amount of non-essential motion shown to them. If the value is set to `reduced` animations and transitions should be reduced to the bare minimum.

A value set to `no-preference` indicated that the user has no preference on reduced motions, allowing all kinds of animations and movement. [3]

4.2.2. Contrast

With `prefers-contrast` the user can specify if he requests content to be presented with a lower or higher contrast. The value `no-preference` indicates that no setting regarding contrast preference was configured by the user.

If the value is set to `more` a higher level of contrast is requested by the user whilst a value of `less` indicated that the opposite, so a lower contrast, is preferred by the user.

Additionally, `custom` can be set as a value as well, meaning the user prefers a specific set of colors. The color palette is typically specified within the media feature `forced-colors` explained in the following section. [4]

4.2.3. Forced Colors

`forced-colors` detects, if a user agent has a forced colors mode enabled such as Windows High Contrast. This media-feature has either the value `active` if such a mode is enabled or `none` if not.

When activated, this overwrites and restricts colors defined in the style sheet. Color values are not affected by styles defined in CSS but instead forced by the browser at paint time. The colors enforced depend on the context of an element. Additionally, backplates are drawn behind text to ensure legibility to preserve contrast for text placed on top of images.

Developers are not encouraged to use this media query to create a separate design for users with this feature enabled but to make small tweaks to improve usability if a certain part of a page would not work well in the forced colors context such as replacing box-shadow stylings with a border to not lose the contrast of a button that is only elevated from the background with a box-shadow. [5]

4.2.4. Reduced Transparency

When a user has enabled a setting to reduce the transparent or translucent layer effects the `prefers-reduced-transparency` media-feature will be set to `reduced` while a value of `no-preference` indicates default behavior is expected by the user. This can help improve contrast and readability for some users.

This feature is restricted available and not yet fully supported by Firefox and Safari.

4.2.5. Color Scheme

A feature more broadly used compared to the others introduced in this chapter is the `prefers-color-scheme`. It indicates if the user requests a light or dark color theme set through the operating system or a user agent setting. Possible values are `light` for a light color theme and `dark` for a dark color theme. [6]

4.2.6. Inverted Colors

Using inverted colors can have unpleasant side effects such as shadows turning into highlights, reducing the readability of the content. Developers can react to this preference and ensure the pages integrity with this media-feature. This feature is only supported by Safari so far. [7]

4.2.7. Accessing Media Features through JavaScript

The `window` interface provides the `matchMedia()` method, returning a `MediaQueryList` object to check if the `document` matches a media query string.

```
// Checks if the system settings prefere a dark color scheme
const isDarkTheme = window.matchMedia("(prefers-color-scheme:
dark)").matches;
```

This enables developers to not only react to preferences within CSS but also consider the users wishes within business logic and web page specific features. [8]

4.2.8. Changes in the World of Media Queries

Continuous development and improvement take place in the world of CSS media queries but there is currently one big visual accessibility concern that is not considered yet. Different types of colorblindness and other visual impairments in regards of color perception fall short. There is an open issue in the W3Cs csswg-drafts repository proposing an additional `media-feature` called `color-vision-adjustment` that intends to cover that area to enhance web accessibility for users with color vision deficiencies.

This proposal intends to cover various specific types of colorblindness such as `protanopia` (red deficiency), `deuteranopia` (red-green deficiency), `tritanopia` (blue-yellow deficiency) or `achromatopsia` (near total color blindness - grayscale) that were covered in the previous P7 project with bookmarklets simulating these visual impairments.

Having such a `media-feature` would increase the awareness as well as the possibilities to directly react to the needs of users affected which such impairments.

4.2.9. Deprecated Features

There were media types considering accessibility needs like `embossed`, `aural` and `braille`, but they got deprecated, assuming distinctions between device types will become more blurred in the future and due to media-type referring to device categories rather than the media they support. [9], [10]

4.2.10. Security Concerns

Data accessible with media queries can be abused to construct a fingerprint, helping to identify and track a device. To prevent this, browsers may fudge returned values in some manners. Some Browsers also provide the possibility to prevent this abuse in the browser settings, such as “Resist Fingerprinting” in Firefox resulting in many media queries only reporting default values rather than device specific settings. [10]

4.3. CSS Custom Properties

4.4. CSS Functions

4.5. Ishihara

5. Implementation

5.1. Preferences Weidget

5.2. Contrast Color Functions

5.3. Interactive Ishihara Plate

6. Discussion

7. Conclusions

Bibliography

[1] Florian Schnidrig, "Accessibility on the Web – Where We Stand." Accessed: Apr. 13, 2026. [Online]. Available: https://accessible-web-initiative.gitbook.io/accessibility-on-the-web-where-we-stand

[2] Mozilla Corporation, "Using media queries." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Media_queries/Using

[3] Mozilla Corporation, "prefers-reduced-motion." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-reduced-motion

[4] Mozilla Corporation, "prefers-contrast." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-contrast

[5] Mozilla Corporation, "forced-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/forced-colors

[6] Mozilla Corporation, "prefers-color-scheme." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[7] Mozilla Corporation, "inverted-colors." Accessed: Mar. 09, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media/prefers-color-scheme

[8] Mozilla Corporation, "Window: matchMedia() method." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/matchMedia

[9] World Wide Web Consortium, "Media Queries – disposition of comments." Accessed: Apr. 16, 2026. [Online]. Available: https://www.w3.org/Style/css3-updates/css3-mediaqueries-comments

[10] Mozilla Corporation, "@media." Accessed: Apr. 16, 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/At-rules/@media#media_types

List of Figures